2026학년도 2학기 인문데이터 수량분석

제13강 공간 데이터

박수민

서울대학교 인문대학 연계전공 인문데이터과학

2026년 11월 2일 월요일

오늘의 목표

1. 경도·위도가 적힌 표를 공간 자료로 바꾸어 지도 위에 점으로 찍을 수 있다.
2. 좌표계와 투영이 무엇인지, 왜 바꾸는지 설명할 수 있다.
3. 경계 파일(GeoJSON)을 읽어 값에 따라 칠한 지도(단계구분도)를 그릴 수 있다.
4. 넓이·길이·거리를 재고, 자리를 기준으로 두 표를 이을 수 있다(공간 조인).

사전 준비

패키지 셋을 설치하고, 지도 타일용 `ggmaptile`은 8강의 `hdir`처럼 GitHub에서 받는다(8강에서 받은 `ggimg`를 그대로 쓴다).

```
install.packages(c("sf", "rnaturalearth", "rnaturalearthdata"))
remotes::install_github("taylor-arnold/rpkg/ggmaptile")
```

1 **점: 좌표를 지도 위에**
경도와 위도
투영

2 면: 경계와 단계구분도

3 넓이, 길이, 거리

4 공간 조인

5 지도 타일

6 마무리

10강이 남긴 숙제

실제 자리 vs. 힘의 평형

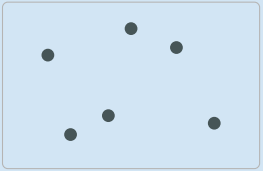
10강에서 Lim(2023)의 그림은 나라를 지도 위 **실제 자리**에 놓았고, 우리는 9강의 방법대로 힘의 평형에 놓았다. “지도 위에 점을 놓는 법은 13강에서”라고 미뤄 두었다.

장소가 붙은 인문학 자료

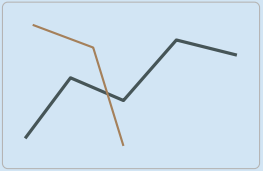
“자료의 80%에는 공간 요소가 있다”는 말이 있다(교재). 도서관 장서 목록에도 출판지와 저자의 출생지가 숨어 있다. 편지의 발신지, 신문 기사의 지명, 사진의 촬영지, 유물의 출토지도 그렇다.

- 오늘은 교재의 미국 대도시권 자료(3–5강)로 도구를 익히고, 10강의 4–5세기 도읍 일곱 곳을 직접 지도에 찍는다.
- 12강 끝에서 약속한 대로 좌표를 지도에 찍고, **지역별로 묶어 센다**.

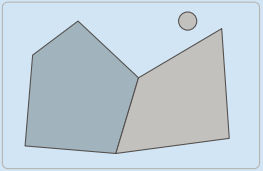
점, 선, 면



점 POINT
도시, 유적, 사찰



선 LINESTRING
도로, 강, 사행로



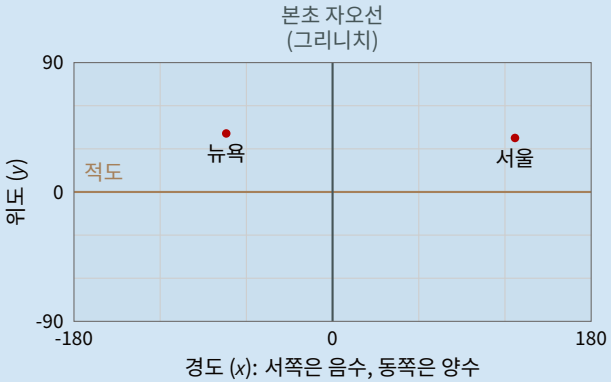
면 POLYGON
나라, 행정구역, 도시권

단순 피처(simple features)

지도 위의 것을 점·선·면과 그 묶음으로 적는 국제 표준. 패키지 `sf`의 이름이 여기서 왔다.

➔ 섬이 여럿인 나라처럼 조각 여러 개가 한 행이면 `MULTIPOLYGON`이다(오른쪽 위의 작은 섬).

경도와 위도



수 두 개로 적는 자리

서울은 (126.98, 37.57), 뉴욕은 $(-74.01, 40.71)$. **경도가 먼저**다. 그림의 x, y와 같은 순서다.

도분초

옛 자료는 37°34′N처럼 도·분·초로 적는다. $37 + 34/60 = 37.57$로 바꾸어 쓴다.

⚠ 말로는 “위도와 경도”라고 위도를 먼저 부르지만, 자료에서는 경도가 먼저다. 이 차이가 실수를 부른다.

오늘의 자료 (1): 미국 대도시권의 중심

```r
library(tidyverse)
library(sf)
cbsa <- read_csv(file.path("hdr", "data", "acs_cbsa.csv"))
cbsa |> select(name, geoid, quad, lon, lat, pop)
```

```
# A tibble: 934 × 6
  name         geoid quad     lon   lat   pop
  <chr>        <dbl> <chr>  <dbl> <dbl> <dbl>
1 New York     35620 NE      -74.1  40.8 20.0
2 Los Angeles 31080 W      -118.   34.2 13.2
3 Chicago      16980 NC      -88.0  41.7  9.61
# i 931 more rows
```

- ➔ 3강부터 쓴 대도시권(CBSA, core-based statistical area) 934곳이다. 교재는 community-based라 적었지만 core-based가 맞다.
- ➔ `lon`, `lat`은 대도시권 **한가운데**의 경도와 위도다. 지금까지는 그냥 수 열로 두었다.
- 💡 `pop`은 백만 명 단위다. `quad`가 `"0"`인 18곳은 알래스카·하와이·푸에르토리코다(4강).

st_as_sf(): 표를 공간 자료로

```
cbsa_geo <- cbsa |>
  st_as_sf(coords = c("lon", "lat"), crs = 4326, remove = FALSE)
cbsa_geo |> select(name, lon, lat, pop)
```

```
Simple feature collection with 934 features and 4 fields
Geometry type: POINT
Dimension:     XY
Bounding box:  xmin: -159.5968 ymin: 17.9943 xmax: -66.08367 ymax: 64.8101
Geodetic CRS:  WGS 84
# A tibble: 934 × 5
  name            lon   lat   pop             geometry
  <chr>         <dbl> <dbl> <dbl>          <POINT [°]>
1 New York      -74.1  40.8 20.0  (-74.10106 40.76877)
2 Los Angeles -118.   34.2 13.2  (-118.1487 34.21941)
# i 932 more rows
```

coords 좌표가 든 열. *x*(**경도**)**를 먼저** 적는다.

crs 좌표계. 4326은 "이 수는 경도·위도다"라는 뜻이다(2절).

remove FALSE면 lon, lat 열을 지우지 않고 남긴다. 뒤에서 쓸모가 있다.

공간 자료의 생김새

```
Geometry type: POINT
Bounding box: xmin: -159.6 ymin: 18.0 ...
Geodetic CRS: WGS 84
```

머리말: 도형의 종류, 자료가 걸친 범위, 좌표계

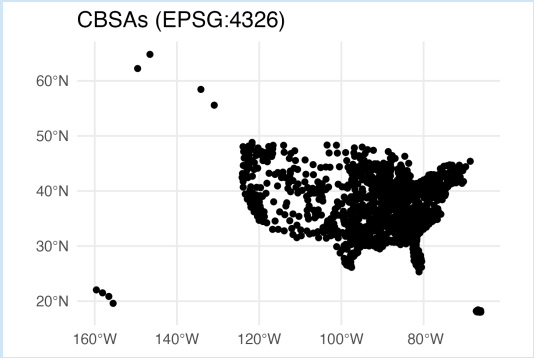
name	lon	lat	pop	geometry
New York	−74.1	40.8	20.0	POINT (-74.1 40.8)
Los Angeles	−118.1	34.2	13.2	POINT (-118.1 34.2)
Chicago	−88.0	41.7	9.61	POINT (-88.0 41.7)

보통의 열 (tibble 그대로) | 도형 열

- 공간 자료(`sf` 객체) = tibble + `geometry` 열 + 좌표계. `filter()` 등이 그대로 듣는다.
- `geometry` 열은 `select()`에서 빼도 **따라다닌다**. 떼어 내려면 `as_tibble()`.

geom_sf(): 첫 지도

```
cbsa_geo |>
  ggplot() +
    geom_sf() +
    labs(title =
      "CBSAs (EPSG:4326)")
```



x, *y*를 주지 않는다

geom_sf()는 geometry 열에서 자리를 읽는다. 점이든 면이든 같은 함수다.

- 축에는 120°W처럼 **도(degree)**가 붙는다. 뜻이 분명해서 축 이름은 지운다(교재).
- 왼쪽 위는 알래스카, 왼쪽 아래는 하와이, 오른쪽 아래 덩어리는 푸에르토리코다.

10강의 도읍을 직접 적기

```
capitals <- tribble(
  ~city,        ~polity,          ~lon,   ~lat,
  "Jiankang",   "Eastern Jin",    118.78, 32.05,    # 건강
  "Pingcheng",  "Northern Wei",   113.30, 40.09,    # 평성
  "Gungnae",    "Goguryeo",       126.18, 41.13,    # 국내성
  "Pyongyang",  "Goguryeo",       125.75, 39.02,    # 평양
  "Hanseong",   "Baekje",         127.12, 37.53,    # 한성
  "Ungjin",     "Baekje",         127.12, 36.46,    # 웅진
  "Geumseong",  "Silla",          129.22, 35.83)    # 금성
capitals_geo <- capitals |>
  st_as_sf(coords = c("lon", "lat"), crs = 4326, remove = FALSE)
```

➔ 동진·북위의 도읍과 고구려·백제·신라의 도읍이다. 좌표는 지금의 유적 자리(풍납토성, 공산성, 월성 등)에서 땄다.

💡 좌표는 위키백과 문서의 오른쪽 위, 지도 서비스의 “이곳의 좌표”, 발굴 보고서에서 찾는다. 지명 목록을 한꺼번에 좌표로 바꾸는 일(지오코딩)은 14강에서 한다.

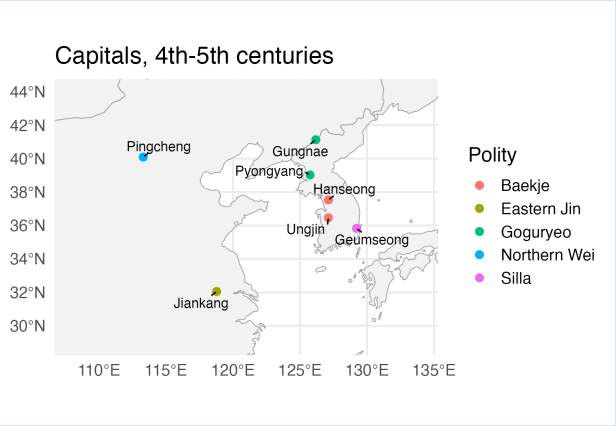
바탕 지도 위에

```
library(rnaturalearth)
library(ggrepel)
world <- ne_countries(scale = 50)    # 나라 경계 242개

ggplot() +
  geom_sf(data = world, fill = "grey95", color = "grey60") +
  geom_sf(aes(color = polity), data = capitals_geo, size = 2.5) +
  geom_text_repel(aes(lon, lat, label = city), data = capitals,
                  size = 3.6, seed = 1, min.segment.length = 0) +
  coord_sf(xlim = c(108, 134), ylim = c(29, 44)) +
  labs(title = "Capitals, 4th-5th centuries",
       color = "Polity", x = NULL, y = NULL)
```

- 층 둘을 겹친다. 바탕은 나라의 면, 위는 도읍의 점. `data`를 층마다 따로 준다.
- `coord_sf()`로 보여 줄 범위(경도 108–134, 위도 29–44)를 자른다. 이름표는 3강의 `geom_text_repel()`에 남겨 둔 `lon`, `lat` 열을 쓴다.

4–5세기 동아시아의 도읍



Natural Earth

공개된 세계 지도 자료. `rnaturalearth`가 나라 경계를 불러온다.

바다를 사이에 두고

건강과 한반도 사이에 황해가 있다. 10강의 교섭은 이 바다를 건넜다.

⚠ 바탕의 해안선과 국경은 **지금**의 것이다(4절에서 다시 본다).

경도와 위도를 뒤바꾸면

```
capitals |>
  st_as_sf(coords = c("lat", "lon"), crs = 4326)
```

```
Simple feature collection with 7 features and 2 fields
Geometry type: POINT
Dimension:     XY
Bounding box:  xmin: 32.05 ymin: 113.3 xmax: 41.13 ymax: 129.22
Geodetic CRS:  WGS 84
```

오류가 나지 않는다

객체는 멀쩡히 만들어진다. 거리를 재면 경고 (`bounding box has potentially an invalid value range`) 하나만 뜬다.

그럴듯하게 틀린 값

한성-웅진의 거리가 72 km로 나온다.
실제로는 119 km다.

❗ Bounding box를 본다. `ymin: 113.3`, 곧 위도 113°는 지구에 없다. 새 공간 자료를 만들면 머리말부터 읽는다.

둥근 지구, 평평한 화면

투영(projection)

둥근 지구를 평면에 옮기는 규칙. 넓이, 모양, 거리를 **한꺼번에 지킬 수는 없다**.

좌표계(CRS)

수 두 개가 지구의 어디를 가리키는지 정한 약속. EPSG 번호로 부른다.

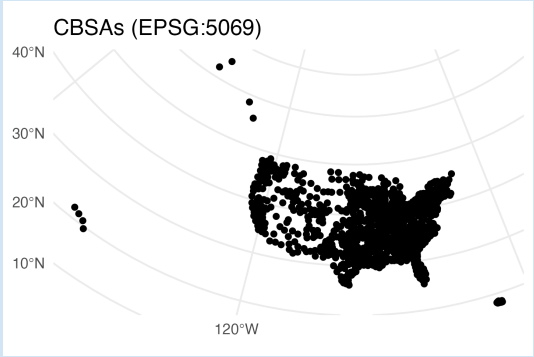
EPSG	이름	단위	쓰임
4326	WGS 84 (경위도)	도	GPS와 웹 지도의 좌표
3857	웹 메르카토르	미터	지도 타일(5절)
5069	미국 본토 앨버스 등적	미터	미국 지도. 넓이를 지킨다
5179	한국 통합좌표계(UTM-K)	미터	국내 경계 파일

➔ 4326은 경위도를 그대로 x, y로 쓴다. 극지방으로 갈수록 땅이 옆으로 늘어난다.

💡 자료가 있는 나라나 지역의 이름으로 `https://epsg.io`에서 번호를 찾는다(교재).

st_transform(): 투영 바꾸기

```
cbsa_geo |>
  st_transform(5069) |>
  ggplot() +
    geom_sf() +
    labs(title =
      "CBSAs (EPSG:5069)")
```



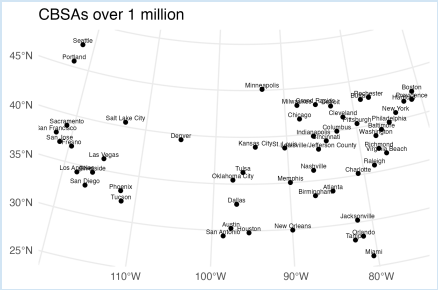
바꾼 것은 한 줄

앞 장의 코드에 `st_transform(5069)`를 넣었다 (제목도 바꿨다).

- 경선과 위선이 **휘었다**. 지구본을 돌려 미국을 정면에서 본 모습이다.
- 본토에 맞춘 투영이라 멀리 떨어진 알래스카와 하와이는 기울어진다.

투영하면 단위가 미터가 된다

```
cbsa_geo |>
  filter(quad != "0",
         pop >= 1) |>
  st_transform(5069) |>
  ggplot() +
    geom_sf() +
    geom_sf_text(
      aes(label = name),
      nudge_y = 50000,
      size = 2.6) +
    labs(title =
      "CBSAs over 1 million",
      x = NULL, y = NULL)
```



➔ `nudge_y = 50000`: **50 km** 위로. 5069는 미터 단위다. 4326이면 5만 **도**다.

💡 `x = NULL, y = NULL`: `geom_sf_text()`가 되살린 축 이름을 지운다(교재).

한국 자료의 좌표계: 5179

```
capitals_geo |>
  st_transform(5179) |>
  st_coordinates()
```

```
              X       Y
[1,]   175401.3 1373620
[2,] -212558.9 2329792
[3,]   889203.0 2348213
[4,]   848502.2 2114638
[5,]   966425.2 1947922
[6,]   965951.4 1829223
[7,] 1155358.9 1760644
```

미터로 적은 자리

북위 38°, 동경 127.5°를 (1,000,000, 2,000,000)으로 두고 동쪽·북쪽으로 몇 m인지 적는다. 5행 한성은 원점에서 서쪽으로 약 34 km, 남쪽으로 약 52 km다.

한반도 밖은 범위 밖

2행 평성의 X가 음수다. 이 좌표계는 한국에 맞춘 것이다.

⚠ 통계지리정보서비스(SGIS) 등에서 받는 국내 경계 파일은 이런 미터 좌표다. 경위도 점과 겹치려면 `st_transform()`으로 한쪽 좌표계에 맞춘다.

1 점: 좌표를 지도 위에

2 **면: 경계와 단계구분도**
경계를 담은 파일
단계구분도

3 넓이, 길이, 거리

4 공간 조인

5 지도 타일

6 마무리

GeoJSON 파일 들여다보기

```
{
"type": "FeatureCollection",
"crs": { ..., "properties": { "name": "urn:ogc:def:crs:OGC:1.3:CRS84" } },
"features": [
{ "type": "Feature",
  "properties": { "name": "Maine", "abb": "ME", "fips": "23" },
  "geometry": { "type": "MultiPolygon", "coordinates": [ [ [
      [ -70.703818, 43.059825 ], [ -70.8268, 43.127086 ], ...,
      [ -70.703818, 43.059825 ] ] ] ] } },
{ "type": "Feature", "properties": { "name": "New Hampshire", ...
```

- ➔ `hdr/data/geo_state.geojson`을 편집기로 연 모습(줄여 적었다). 6강의 JSON이다. `properties`는 열이 되고 `geometry`는 좌표의 목록이 된다.
- ➔ 면은 꼭짓점을 차례로 이은 고리다. **첫 점과 끝 점이 같아서** 고리가 닫힌다.
- 💡 CRS84는 경도·위도(4326과 같은 값, 경도가 먼저)다. 셰이프 파일(`.shp`와 짝 파일들)도 같은 함수로 읽는다.

read_sf(): 경계 읽기

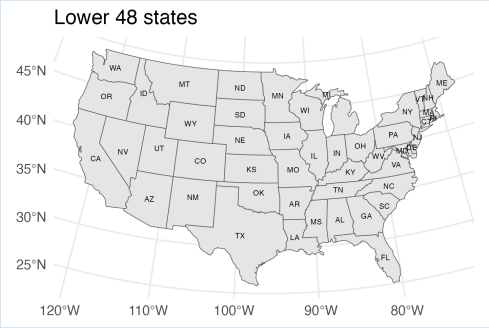
```
1 state <- read_sf(file.path("hdr", "data", "geo_state.geojson"))
2 state
```

```
Simple feature collection with 50 features and 3 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -168.1289 ymin: 18.91747 xmax: -66.9824 ymax: 71.29634
Geodetic CRS:  WGS 84
# A tibble: 50 × 4
  name           abb   fips                                          geometry
  <chr>          <chr> <chr>                                  <MULTIPOLYGON [°]>
1 Maine          ME    23    (((-70.70382 43.05983, -70.8268 43.12709, -70.81787…
2 New Hampshire  NH    33    (((-71.50109 45.01338, -71.40564 45.19814, -71.2844…
# i 48 more rows
```

- ➔ read_csv() 대신 read_sf(). 읽자마자 공간 자료다. st_as_sf()가 필요 없다.
- ➔ 50개 주가 모두 MULTIPOLYGON이다. 섬이 많은 하와이 같은 주가 섞여 있기 때문이다.

주의 경계와 이름

```
state48 <- state |>
  filter(!(abb %in%
    c("AK", "HI")))
state48 |>
  st_transform(5069) |>
  ggplot() +
    geom_sf() +
    geom_sf_text(
      aes(label = abb),
      size = 2.4) +
    labs(title =
      "Lower 48 states",
      x = NULL, y = NULL)
```



➔ `geom_sf_text()`는 이름표를 **면의 한가운데**에 놓는다. 알래스카·하와이를 뺀 본토 48개 주는 `state48`에 담아 뒤에서도 쓴다.

대도시권의 경계: 이름이 없다

```
cbsa_reg <- read_sf(file.path("hdr", "data", "acs_cbsa_geo.geojson"))
cbsa_reg
```

```
Simple feature collection with 939 features and 3 fields
Geometry type: MULTIPOLYGON
# A tibble: 939 × 4
  geoid quad    pop                                                  geometry
  <dbl> <chr> <dbl>                                       <MULTIPOLYGON [°]>
1 35620 NE     20.0  (((-72.03683 41.24984, -72.03496 41.25546, -72.03374 41.257…
2 31080 W      13.2  (((-118.6044 33.47855, -118.5988 33.47794, -118.5859 33.473…
# i 937 more rows
```

➔ 대도시권의 실제 경계(면)다. 열은 geoid, quad, pop뿐이고 **이름이 없다**. 이름은 cbsa에서 geoid를 키로 붙인다(5강).

⚠ name은 키가 될 수 없다. Columbus라는 대도시권이 다섯이다(5강의 “기본키가 아니다”).

💡 939곳이다. 점 자료(934곳)에 없는 geoid가 5개 있다.

단계구분도: 값으로 칠하기

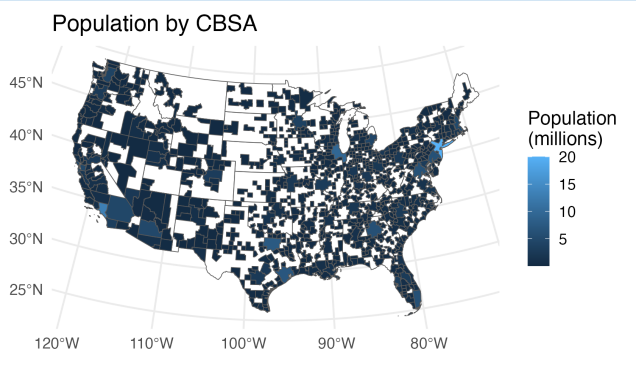
```
cbsa_reg |>
  filter(quad != "0") |>
  st_transform(5069) |>
  ggplot() +
    geom_sf(data = state48, fill = "white") +    # 바탕
    geom_sf(aes(fill = pop)) +                   # 위
    labs(title = "Population by CBSA",
         fill = "Population\n(millions)")
```

층 둘

바탕에 흰 주를 깔고 그 위에 대도시권을 칠한다. `data =`로 층마다 다른 자료를 준다.

- ➔ 좌표계는 맨 위 자료(5069)를 따른다. `state48`은 바꾸지 않아도 된다(교재).
- ➔ `fill`은 면의 안쪽 색, `color`는 경계선의 색이다.
- 💡 교재는 `state48` 자리에 `filter()`를 그대로 적었다.

한 도시가 색을 다 차지한다



단계구분도(choropleth)

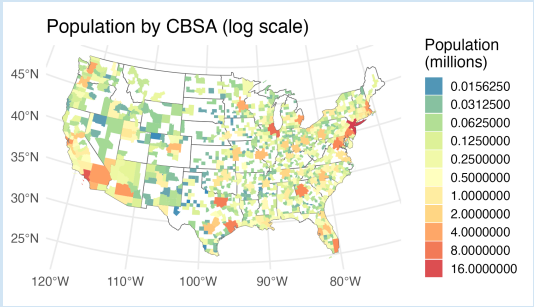
면을 값에 따라 칠한 지도.
선거 결과 지도가 대표적이다.

➔ 뉴욕, 로스앤젤레스, 시카고는 알아볼 수 있다(교재). 나머지는 거의 다 짙은 남색이다.

⚠ 뉴욕 하나(2천만 명)가 색의 범위를 다 차지해서, 작은 값들의 차이가 한 색에 묻혔다.

로그 척도로 칠하기

```
geom_sf(aes(fill = pop),
        color = NA) +
scale_fill_distiller(
  trans = "log2",
  palette = "Spectral",
  guide = "legend",
  n.breaks = 10) +
```



- 앞 장의 `geom_sf(aes(fill = pop))` 한 줄을 왼쪽 코드로 바꾸었다.
- `"log2"`: 색이 한 칸 바뀔 때마다 인구가 **두 배**다. 큰 도시와 작은 도시가 함께 읽힌다.
- `color = NA`: 경계선을 지운다. `"Spectral"`: 양 끝이 다른 색이다.

이 지도는 무엇을 보여 주는가

넓은 면이 눈을 끈다

캘리포니아의 리버사이드 대도시권은 71,001 km^2 다. 남한 넓이(약 10만 km^2)의 70%에 이른다. 인구가 같아도 넓은 면이 더 크게 보인다.

흰 곳

대도시권에 들지 않은 곳이지 사람이 없는 곳이 아니다.

공간 분석가들의 농담(교재)

"무언가 새로운 것을 보여 준다는 지도는 대부분 그냥 인구 지도다."

교재의 제안

인구 대신 **인구 밀도**로 칠하는 편이 나을 수도 있다. 그러려면 넓이가 필요하다(3절).

➔ 넓이·인구 같은 기준으로 나눈 값, 그리고 색의 구간을 나누는 방법은 14강에서 다룬다.

1 점: 좌표를 지도 위에

2 면: 경계와 단계구분도

3 **넓이, 길이, 거리**
넓이와 중심점
길이와 거리

4 공간 조인

5 지도 타일

6 마무리

st_area(): 넓이

```
cbsa_reg |>
  mutate(area = as.numeric(st_area(geometry)) / 1e6) |>
  arrange(desc(area)) |>
  as_tibble() |>
  left_join(select(cbsa, geoid, name), by = "geoid") |>
  select(name, pop, area)
```

```
# A tibble: 939 × 3
  name          pop   area
  <chr>       <dbl>  <dbl>
1 Riverside 4.58    71001.
2 Anchorage 0.399   68795.
3 Elko      0.0550  55310.
# ℹ 936 more rows
```

단위가 붙은 수

st_area()는 [m^2] 단위가 붙은 값을 준다. as.numeric()으로 떼고 1e6으로 나누면 km^2다. 경위도 자료도 구면 위에서 잰다.

➔ 이름은 geoid로 붙였다. 교재처럼 작은 순으로 놓으면 푸에르토리코의 대도시권이 앞선다.

st_centroid(): 중심점

```
cbsa_reg$geometry |>
  st_centroid() |>       # 면마다 한가운데의 점
  st_coordinates() |>    # X는 경도, Y는 위도
  as_tibble() |>
  rename(lon = X, lat = Y) |>
  bind_cols(cbsa_reg)
```

```
# A tibble: 939 × 6
    lon   lat geoid quad    pop                                   geometry
  <dbl> <dbl> <dbl> <chr> <dbl>                             <MULTIPOLYGON [°]>
1 -74.1  40.8 35620 NE     20.0  (((-72.03683 41.24984, -72.03496 41.25546, -…
2 -118.  34.2 31080 W      13.2  (((-118.6044 33.47855, -118.5988 33.47794, -…
# ℹ 937 more rows
```

➔ 1절 cbsa의 lon, lat과 같다(차이 $< 0.00001°$). 점 자료는 **면의 중심점**이었다.

➔ 면이 전체에 비해 작으면 점 하나로 다루는 편이 간단하다(교재).

⚠ 중심점이 면 **밖**에 떨어지기도 한다. 초승달이나 섬이 여럿인 면이 그렇다(4절).

st_length(): 길이

```
roads <- read_sf(file.path(
  "hdr", "data",
  "geo_ny_roads.geojson"))
roads |>
  mutate(length = as.numeric(
    st_length(geometry))) |>
  filter(!is.na(name)) |>
  group_by(name) |>
  summarize(length =
    sum(length)) |>
  arrange(desc(length)) |>
  as_tibble()
```

```
# A tibble: 945 × 3
  name                length
  <chr>                <dbl>
1 State Rte 9a        42412.
2 F D R Dr            35189.
3 Henry Hudson Pkwy   24373.
4 Broadway            22615.
5 Riverside Dr        16571.
# i 940 more rows
```

선(LINESTRING)

맨해튼 도로 조각 2,238개. 이름이 같은 조각의 길이를 더했다.

➔ 단위는 미터다. 섬의 서쪽(9A)과 동쪽(FDR)을 따라 남북으로 달리는 길이 가장 길다. 출력의 셋째 열(geometry)은 지면에서 뺐다.

st_distance(): 도읍 사이의 거리

```
mat <- as.matrix(st_distance(
  capitals_geo$geometry, capitals_geo$geometry))
cap_dist <- tibble(
  city_start = capitals_geo$city[row(mat)],    # 칸의 행 번호
  city_end = capitals_geo$city[col(mat)],      # 칸의 열 번호
  distance = as.numeric(mat) / 1000)    # km
cap_dist <- filter(cap_dist, city_start < city_end)
cap_dist |> arrange(distance)
```

```
# A tibble: 21 × 3
  city_start city_end  distance
  <chr>      <chr>        <dbl>
1 Hanseong   Ungjin        119.
2 Geumseong  Ungjin        201.
3 Hanseong   Pyongyang     204.
# i 18 more rows
```

교재의 방법

7×7 거리 행렬을 긴 표로 편다. `city_start < city_end`로 자기 자신과 뒤집힌 쌍을 지워 21쌍이 남는다.

- 한성–건강은 975 km. **직선(대원) 거리**다. 사신은 해안을 따라, 바람을 기다려 갔다. 실제 길은 이보다 길다.

가장 외딴 대도시권

```
index <- st_nearest_feature(cbsa_geo)
cbsa_geo$name_close <- cbsa_geo$name[index]
cbsa_geo$distance <- as.numeric(st_distance(
  cbsa_geo$geometry, cbsa_geo$geometry[index], by_element = TRUE))
cbsa_geo |>
  as_tibble() |>
  arrange(desc(distance)) |>
  select(name, name_close, distance)
```

```
# A tibble: 934 × 3
  name       name_close distance
  <chr>      <chr>         <dbl>
1 Juneau     Ketchikan   374643.
2 Ketchikan  Juneau      374643.
3 Anchorage  Fairbanks   321784.
# i 931 more rows
```

교재 코드 그대로

`st_nearest_feature()`는 가장 가까운 다른 점의 행 번호를 준다. `by_element = TRUE`: 1번은 1번끼리 잰다.

➔ 주노(알래스카)는 가장 가까운 대도시권이 375 km 밖이다. 교재는 가까운 순으로 보였다(푸에르토리코, 14 km).

1 점: 좌표를 지도 위에

2 면: 경계와 단계구분도

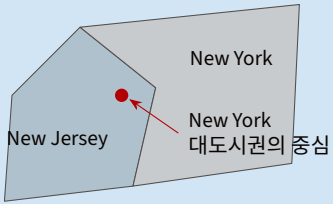
3 넓이, 길이, 거리

4 **공간 조인**
점이 든 면
이웃과 거리

5 지도 타일

6 마무리

공간 조인: 키 대신 자리



name	name_state	abb
New York	New Jersey	NJ

공간 조인(spatial join)

5강의 조인은 **키가 같은** 행끼리 이었다. 공간 조인은 **도형의 관계**로 잇는다. 기본은 "겹친다(점이 면 안에 있다)"이다.

```
1 # 교재가 만든 도우미 함수
2 spatial_join <- function(...) {
3   return(st_as_sf(
4     as_tibble(st_join(...))))
5 }
```

➔ `st_join()`의 결과는 tibble 모양이 아니다. 교재는 tibble로 바꾼 뒤 다시 공간 자료로 만드는 함수를 정의해 쓴다. 함수를 만드는 법은 교재 11장의 내용이니, 여기서는 이 정의를 한 번 실행해 두기만 한다.

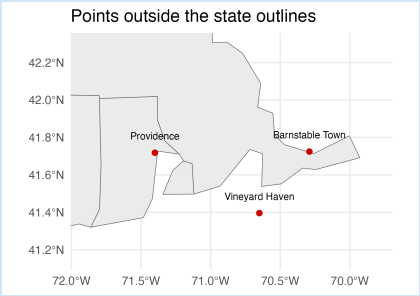
점이 든 주 찾기

```
cbsa_geo |>
  spatial_join(state, suffix = c("", "_state")) |>
  select(name, name_state, abb)
```

```
Simple feature collection with 934 features and 3 fields
# A tibble: 934 × 4
  name        name_state abb               geometry
  <chr>       <chr>      <chr>          <POINT [°]>
1 New York    New Jersey NJ    (-74.10106 40.76877)
2 Los Angeles California CA    (-118.1487 34.21941)
3 Chicago     Illinois   IL    (-87.95882 41.70061)
# i 931 more rows
```

- ➔ 두 표 모두 name 열이 있다. suffix로 오른쪽 표의 것에 _state를 붙였다(5강의 조인과 같다).
- ➔ 뉴욕 대도시권은 세 주에 걸쳐 있는데, 그 **중심점**이 허드슨강 건너 뉴저지에 떨어졌다.
- ❗ 남는 도형은 **왼쪽**(파이프 앞) 표의 것이다. 결과는 여전히 934개의 점이다.

어느 주에도 없는 점 14개




```
cbsa_geo |>
  spatial_join(state) |>
  filter(is.na(abb)) |>
  nrow()
```

```
[1] 14
```

10개 + 4개

10개는 푸에르토리코다. 주가 아니어서 `state`에 없다.

⚠ 4개는 중심점이 **주 경계 밖**에 떨어졌다. 단순한 경계 파일에 마서스비니어드 섬(Vineyard Haven)이 없다.

➔ 짝이 없는 점은 지우지 않고 NA로 남긴다(왼쪽 조인). 안쪽 조인은 `left = FALSE`.

주마다 몇 곳인가

```
state_n <- cbsa_geo |>
  spatial_join(state,
    suffix = c("", "_state")) |>
  as_tibble() |>
  count(abb, sort = TRUE)
state_n
```

```
# A tibble: 51 × 2
  abb       n
  <chr> <int>
1 TX       69
2 OH       44
3 NC       38
# i 48 more rows
```

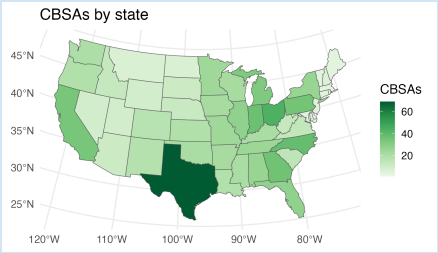
방향을 바꾸면

`state |> spatial_join(cbsa_geo)`는 920행($= 934 - 14$)이다. 주의 도형이 대도시권 수만큼 **되풀이**된다(캘리포니아 모양이 수십 벌).

- ➔ 교재의 권고: 점 쪽에서 잇고, `as_tibble()`로 도형을 뗀 뒤 센다.
- ➔ 12강에서 약속한 "지역별로 묶어 세기"다. 주가 없는 14곳은 NA로 세어진다.

세어서 칠하기

```
state48 |>
  left_join(state_n,
            by = "abb") |>
  st_transform(5069) |>
  ggplot() +
    geom_sf(aes(fill = n)) +
    scale_fill_distiller(
      palette = "Greens",
      direction = 1) +
    labs(title =
      "CBSAs by state",
      fill = "CBSAs")
```



➔ 공간 조인으로 세고, 5강의 키 조인(abb)으로 주의 면에 붙여 칠했다.

❗ 텍사스는 도시가 많아서 짙은가, **땅이 넓어서** 짙은가? 나누어 보아야 안다(14강).

맞닿은 주: st_touches

```
state |>
  spatial_join(
    state,
    join = st_touches,
    suffix = c("", "_out")) |>
  as_tibble() |>
  select(abb, abb_out)
```

```
# A tibble: 216 × 2
  abb   abb_out
  <chr> <chr>
1 ME    NH
2 NH    ME
3 NH    VT
4 NH    MA
5 DE    PA
# i 211 more rows
```

join =

어떤 관계로 이을지 정한다. st_touches는 **경계를 맞댄** 쌍이다.

- 같은 표를 자기 자신과 잇는다. 메인은 뉴햄프셔 하나와만 맞닿는다.
- 216행 = 107쌍 × 2 + HI, AK의 NA. 미주리·테네시가 8개로 가장 많다.

100 km 안의 대도시권

```
cbsa_geo |>
  spatial_join(
    cbsa_geo,
    join = st_is_within_distance,
    dist = 100000,
    suffix = c("", "_out")) |>
  as_tibble() |>
  select(name, name_out)
```

```
# A tibble: 5,586 × 2
  name        name_out
  <chr>       <chr>
1 New York    New York
2 New York    Bridgeport
3 New York    Poughkeepsie
4 New York    Trenton
5 Los Angeles Los Angeles
# i 5,581 more rows
```

➔ `dist`의 단위는 미터다. 100,000 m = 100 km.

⚠ 자기 자신도 거리 0이라 짝이 된다(1행). 934행이 그런 것이니 `filter(name != name_out)`로 지운다. 다만 이름이 겹치는 대도시권(Columbus 등)이 있어 `geoid`로 거르는 편이 안전하다.

지금의 국경으로 묶으면

```
capitals_geo |>
  spatial_join(
    select(world,
           country = name)) |>
  as_tibble() |>
  select(city, polity, country)
```

```
# A tibble: 7 × 3
  city       polity        country
  <chr>      <chr>         <chr>
1 Jiankang   Eastern Jin   China
2 Pingcheng  Northern Wei  China
3 Gungnae    Goguryeo      China
4 Pyongyang  Goguryeo      North
↪  Korea
5 Hanseong   Baekje        South
↪  Korea
6 Ungjin     Baekje        South
↪  Korea
7 Geumseong  Silla         South
↪  Korea
```

select()로 이름 바꾸기

world의 열 169개 중 name만 country로 이름을 바꿔 가져왔다.

⚠ 국내성(지금의 지린성 지안)이 China로 묶였다. 고구려 도읍이 "중국"에 든다.

❗ 공간 조인의 답은 **쓴 경계만큼만** 옳다. 과거를 물으려면 그 시기의 경계가 필요하다.

1 점: 좌표를 지도 위에

2 면: 경계와 단계구분도

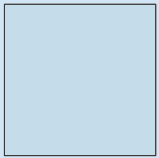
3 넓이, 길이, 거리

4 공간 조인

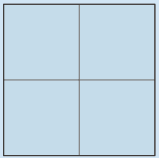
5 **지도 타일**

6 마무리

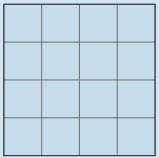
지도 타일: 그림으로 된 지도



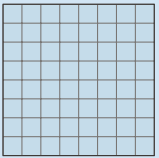
확대 단계 0: 1장



확대 단계 1: 4장



확대 단계 2: 16장



확대 단계 3: 64장

벡터와 래스터

지금까지의 점·선·면은 좌표로 적은 **벡터**다. 타일은 픽셀로 된 **래스터**, 곧 그림이다(15강의 이미지).

타일 서버

확대 단계마다 세계 지도를 $2^z \times 2^z$장의 정사각형 그림으로 잘라 둔다. 웹 지도를 끌 때 조각이 하나씩 채워지는 까닭이다.

➔ 경계 파일을 따로 구하지 않아도 세계 어디든 지형, 도로, 지명이 그려진 바탕을 깔 수 있다(교재).

타일은 어디서 오는가

```
library(ggmaptile)
options(mt_cache_dir = file.path("hdr", "geocache"))
# Sys.setenv(GGMAP_API_KEY = "...")   # 새 지역을 그릴 때만
french_city <- read_csv(
  file.path("hdr", "data", "geo_french_city.csv"))
paris_metro <- read_csv(
  file.path("hdr", "data", "geo_paris_metro.csv"))
```

키와 저장해 둔 타일

타일은 Stadia Maps 서버에서 오고, 무료 계정의 키가 필요하다(교재). `hdr/geocache`에 교재 그림의 타일이 있어 **키 없이** 돈다.

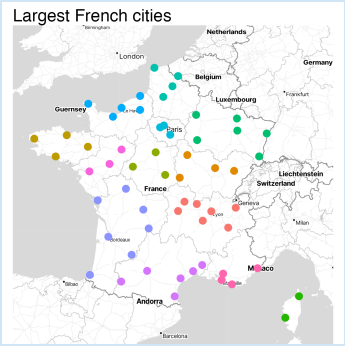
교재와 다른 점

교재의 `ggmap_api_key()`는 지금의 `ggmaptile`에 없다. 키는 환경 변수 `GGMAP_API_KEY`로 넘긴다.

타일 그림은 CC BY 4.0이다. Stamen Maps, OpenMapTiles, OpenStreetMap 기여자를 출처로 밝힌다.

프랑스의 큰 도시들

```
french_city |>
  ggplot(aes(lon, lat)) +
    stat_maptiles(
      zoom_adj = -1) +
    geom_point(
      aes(color = admin_name),
      size = 3,
      show.legend = FALSE) +
    theme_void() +
    mapview() +
    labs(title =
      "Largest French cities")
```

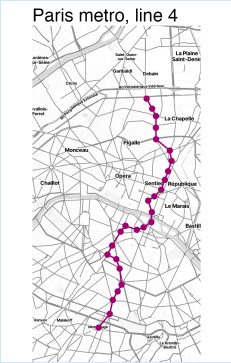


➔ 공간 자료로 바꾸지 않고 `lon`, `lat`을 x, y에 준다. 점의 색은 지역(레지옹)이다.

➔ `zoom_adj = -1`: 한 단계 덜 확대한다. `mapview()`: 여백과 축 이름을 없앤다.

파리 지하철 4호선

```
filter(paris_metro, line == 4) |>
  ggplot(aes(lon, lat)) +
    stat_maptiles(zoom_adj = -2) +
    geom_point(
      aes(color = line_color)) +
    geom_segment(aes(
      xend = lon_end,
      yend = lat_end,
      color = line_color)) +
    scale_color_identity() +
    theme_void() +
    mapview() +
    labs(title =
      "Paris metro, line 4")
```



➔ `geom_segment()`: 다음 역까지 선. `scale_color_identity()`: 적힌 색 그대로.

1 점: 좌표를 지도 위에

2 면: 경계와 단계구분도

3 넓이, 길이, 거리

4 공간 조인

5 지도 타일

6 **마무리**

지도 앞에서 물을 것

오늘 만난 함정

1	경도와 위도의 순서는 맞는가?	(위도 113°)
2	어느 좌표계인가? 단위는 도인가, 미터인가?	(`nudge_y = 50000`, 5179)
3	경계 자료는 얼마나 자세한가?	(주 밖에 떨어진 점 4개)
4	무엇을 키로 잇는가?	(Columbus 다섯 곳, `geoid`)
5	넓어서 짙은가, 많아서 짙은가?	(리버사이드, 텍사스)
6	어떤 거리이고, 어느 때의 경계인가?	(직선 975 km, 국내성)

→ 교재의 말: 무엇이 가깝고 무엇이 먼지를 잴 수 있다는 것 자체가 인문 자료를 탐색하는 길이다. 공간 분석은 지도 그리기에서 끝나지 않는다.

오늘의 함수

함수	하는 일	패키지
`st_as_sf()`	경도·위도 열이 있는 표를 공간 자료로	sf
`read_sf()`	GeoJSON·셰이프 파일 읽기	sf
`st_transform()`	좌표계(투영) 바꾸기	sf
`geom_sf()`·`geom_sf_text()`	도형과 이름표 그리기	ggplot2
`scale_fill_distiller()`	면의 색 척도 (로그 변환)	ggplot2
`st_area()`·`st_length()`	넓이·길이 (단위가 붙는다)	sf
`st_centroid()`·`st_coordinates()`	중심점, 좌표 꺼내기	sf
`st_distance()`·`st_nearest_feature()`	거리, 가장 가까운 것	sf
`st_join()` (`st_touches` 등)	공간 조인	sf
`ne_countries()`	세계 나라 경계	rnaturalearth
`stat_maptiles()`·`mapview()`	지도 타일 깔기	ggmaptile

💡 sf 치트시트 — `https://rstudio.github.io/cheatsheets/sf.pdf`

오늘 배운 것

점

- `st_as_sf()`: 경도가 먼저
- 4326은 경위도, 투영하면 미터
- 바탕 지도 위에 층으로 겹치기

면

- GeoJSON을 `read_sf()`로
- 단계구분도와 로그 척도

재기

- 넓이·길이·거리는 미터 단위
- 점 자료는 면의 중심점이었다

잇기

- 공간 조인: 왼쪽의 도형이 남는다
- 공간 조인으로 세고 키 조인으로 칠한다
- 답은 경계 자료만큼만 옳다

확인 질문

스스로 답해 보기

1. `coords = c("lat", "lon")`으로 잘못 적으면 무슨 일이 일어나는가? 어디를 보면 알아챌 수 있는가?
2. `nudge_y = 50000`은 4326과 5069에서 각각 얼마만큼 옮기라는 뜻인가?
3. `cbsa_reg`에 이름을 붙일 때 `name`이 아니라 `geoid`를 키로 쓰는 까닭은?
4. 점 $\rightarrow$ 주 공간 조인에서 14곳이 NA가 된 까닭 두 가지는?
5. `cbsa_geo |> spatial_join(state)`와 `state |> spatial_join(cbsa_geo)`는 행 수와 도형이 어떻게 다른가?
6. 국내성이 `China`로 묶인 결과를 연구에 쓸 때 무엇을 조심해야 하는가?

💡 답이 막히는 항목이 있으면 eTL Q&A 게시판에 질문을 올린다 (참여 점수 1점).

다음 시간

- 14강 (2026-11-04 수) — 공간 데이터 사례 연습
  - 권오영 (2017), 「韓半島에 輸入된 琉璃구슬의 變化過程과 經路: 初期鐵器–原三國期를 중심으로」, 『湖西考古學』 37, 38–69쪽.
  - 기원전 4세기–기원후 3세기에 들어온 유리구슬을 화학 조성(납-바륨, 포타쉬, 소다)에 따라 세 단계로 나누고 단계마다 다른 길을 그렸다.
  - 논문의 유적을 좌표로 바꾸어(지오코딩) 오늘의 도구로 단계별 지도를 그린다.

더 보고 싶다면

- Lovelace, Nowosad & Muenchow, *Geocomputation with R* (2판) — `https://r.geocompx.org/`
- Pebesma & Bivand, *Spatial Data Science: With Applications in R* (2023) — `https://r-spatial.org/book/`